

تقرير احترافي لتوثيق مشروع رؤية حاسوبية وروبوتكس مع ROS2 و YOLO

هذا التقرير يوثق ما تم تنفيذه في المشروع من بداية العمل حتى نهايته اعتمادًا على محتوى المحادثة المرفقة، ويشمل مراحل التطوير، الحزم المستخدمة، الملفات التي تم إنشاؤها أو استخدامها، مرحلتى التدريب، المشكلات [file:1] التي ظهرت أثناء التنفيذ وحلولها، وأوامر الطرفية التي استُخدمت لتشغيل النظام وبنائه واختباره [code_file:2].

نظرة عامة

يهدف اكتشاف Ubuntu 22.04 و ROS2 Humble المشروع تمحور حول بناء نظام رؤية حاسوبية مدمج مع لاستخراج الإحداثيات الثنائية Astra Pro وربطه بقراءات العمق من كاميرا YOLOv8 باستخدام lever/ذراع تمهيدًا لاستخدامها داخل نظام ذراع روبوتية أو مسار تحكم RViz و TF والثلاثية الأبعاد X, Y, Z ، ثم دمج النتائج مع [file:1] أعلى.

ROS2 ثم انتقل إلى نسخة YOLO، التطور العام للمشروع مرّ من نسخة أولية مستقلة تعمل مباشرة بباينون و والمعايرة، TF broadcasting، و launch files مهيكلة داخل باكدج مخصصة، ثم أضيفت طبقات التكامل مثل RViz [file:1] و فلتر الععمق، والدعم البصري داخل.

التسلسل الزمني للمشروع

البداية وربط الكاميرا

وملفات orbbeccamera باستخدام حزمة ROS2 عبر Orbbec/Astra Pro بدأ العمل بمحاولة تشغيل كاميرا point cloud وإغلاق الـ FPS مثل تقليل دقة و USB الخاصة بها، مع اختبار أكثر من إعداد لتقليل استهلاك launch [file:1] [code_file:2].

أثناء تهيئة الجهاز، كما ظهر ارتباك Device response with bad magic ظهر ميكّرًا خطأ متكرر من الدرايفر وهو astra.launch.py و astrapro.launch.py [file:1] الخاصة بأجهزة متعددة مثل launch بين ملفات [code_file:2].

الانتقال إلى الدرايفر الأنسب

والمناسب أكثر OpenNI2 الميني على astracamera اتجه العمل إلى تجربة orbbeccamera بعد تعثّر مسار ثم عبر السورس عندما لم تتوافر الحزمة الجاهزة كما هو متوقع apt مع محاولة التثبيت عبر Astra Pro، لسلسلة [file:1].

من جديد حتى workspace وإعادة تحميل القواعد، وبناء الـ udev، تمت أيضًا خطوات داعمة مثل تثبيت قواعد متاحة للاشتراك من camera/color/image_raw و camera/depth/image_raw أصبح مواضيع الكاميرا مثل [file:1] [code_file:2]. نود الرؤية

بناء نود الرؤية

ثم ،YOLO بعد استقرار مسار الكاميرا، تم تطوير نود بايثون تستقبل صورة اللون وصورة العمق ، وتشغل نموذج الفعلية بالمليمتر، وعُرضت النتائج على الإطار المرئي مع Z تربط نقطة الاكتشاف بإحداثيات العمق لحساب قيمة [file:1].

محددة تمثل موضع keypoint لاستخراج pose estimation تقليدي إلى detection لاحقًا تطور المنطق من مجرد [file:1] الذراع بدقة أعلى، ثم استخدام هذه النقطة في حساب العمق وإرسال النتائج لطبقات أعلى من النظام

RViz وTF وROS2 دمج

مع ملفات التهيئة القياسية، ثم أضيف ملف levervisionmodel اسمها ROS2 package تم تحويل الحل إلى وإطارات أخرى مثل camera_link وbase_link بين static transforms لتشغيل المكونات، وأضيفت launch arm_base_link [file:1][code_file:2].

ضمن شجرة RViz بحيث يظهر في lever_target ديناميكي للهدف المكتشف مثل Transform كما جرى بث إلى مرحلة الإدماج الفعلي داخل بنية OpenCV وهو ما نقل المشروع من مرحلة الرصد المحلي داخل نافذة TF، [file:1]. روبوتية قابلة للتخطيط والتحكم

الحزم والمكتبات المستخدمة

الدور داخل المشروع	الحزمة/المكتبة
[file:1] TF وال launch وال topics بيئة التشغيل الأساسية للتقيد وال	ROS2 Humble
[file:1][code_file:2] بلغة بايثون ROS2 بناء نودات	rclpy
[file:1][code_file:2] استقبال رسائل الصور من الكاميرا	sensor_msgs
[file:1] OpenCV إلى ROS تحويل رسائل الصور من	cv_bridge
[file:1][code_file:2] عرض الصورة، الرسم على الإطار، وإظهار النتائج بصريًا	OpenCV (cv2)
[file:1][code_file:2] التعامل مع المصفوفات وبيانات العمق وبعض المعالجات العددية	numpy
[file:1][code_file:2] والتعامل مع ملفات الأوزان YOLOv8 / YOLOv8-Pose تحميل وتشغيل	ultralytics
[file:1][code_file:2] ROS2/OpenNI2 عبر Astra Pro تشغيل كاميرا	astracamera
[file:1][code_file:2] تم اختياره في البداية كدرايفر بديل لكنه واجه مشاكل تهيئة للجهاز	orbbeccamera
[file:1][code_file:2] بث التحويلات الثابتة والديناميكية بين الإطارات	tf2_ros
[file:1] PointStamped وTransformStamped تمثيل الإحداثيات/التحويلات مثل	geometry_msgs
[file:1][code_file:2] TF التحقق البصري من الإطارات والهدف المكتشف ضمن شجرة	rviz2
[file:1][code_file:2] والباكدجات workspace بناء الـ	colcon
[file:1][code_file:2] تثبيت الاعتماديات اللازمة للمشروع وباكدجات الكاميرا	rosdep

الملفات التي تم إنشاؤها أو استخدامها

الوظيفة	النوع	الملف
color/depth، نود الرؤية الأساسية؛ تحميل الموديل، الاشتراك في [file:1] حساب موضع الهدف وعمقه، ثم عرض النتيجة أو بثها [code_file:2]	ملف بايثون رئيسي	leverdetector.py
lever [file:1] الخاص بالذراع pose estimation تدريب نموذج الـ [code_file:2]	سكربت تدريب	trainleverpose.py
جمع/تجهيز بيانات إضافية للنسخة الثانية من الداتا أو التدريب [file:1][code_file:2]	سكربت بيانات	collectdatav2.py
[file:1] اختبار مباشر للنموذج أو الكاميرا أثناء التشغيل [code_file:2]	سكربت اختبار مباشر	livetest.py
المساعدة في العثور على الكاميرا أو التحقق من الاتصال/المصدر [file:1][code_file:2]	سكربت تشخيص	findcamera.py
بما يشمل عقدة الرؤية ROS2 تشغيل منظومة المشروع داخل [file:1][code_file:2] اللازمة TFs وربما	Launch ملف	robotbringup.launch.py
[file:1][code_file:2] تعريف الاعتماديات وبيانات الباكج	تعريف ROS package	package.xml
ROS2 [file:1] وتهيئة التثبيت داخل entry points تسجيل [code_file:2]	إعداد باكج بايثون	setup.py
التشغيل/libexec ضبط مسارات السكرينات ومعالجة مشاكل [file:1][code_file:2]	إعدادات التثبيت	setup.cfg
[file:1][code_file:2] جعل المجلد باكج بايثون قابلة للاستيراد	تهيئة بايثون	__init__.py
على الباكج أثناء البناء والتشغيل ROS2 مطلوب لكي تتعرف أدوات [file:1]	مورد ROS2	resource/levervisionmodel
وزن مدرب للنموذج، واستخدم أكثر من ملف best.pt تبعًا لمرحلة [file:1][code_file:2] التدريب	أوزان نموذج	best.pt
pose estimation [file:1] نقطة الانطلاق لتدريب نموذج الـ [code_file:2]	وزن أساسي pretrained	yolov8n-pose.pt

عملية التدريب الأولى

خاص بالمشروع مع تجهيز البيانات داخل Dataset التدريب الأول كان مبنياً على فكرة اكتشاف الذراع بالاعتماد على Roboflow، إلى أن تم الحصول على yolov8n-pose.pt باستخدام وزن ابتدائي YOLOv8-Pose ثم تدريب نموذج، Roboflow، صالح للاختبار best.pt [file:1].

هذه المرحلة مثلت الانتقال من مجرد تشغيل كاميرا وقراءة صور إلى امتلاك نموذج يتعرف على موضع الذراع بصرياً [file:1]. يمكن استخدامها داخل كود الرؤية keypoints ويعيد

عملية التدريب الثانية

التدريب الثاني ظهر في المحادثة على أنه نسخة محسنة مرتبطة ببيانات أو إعدادات أحدث، ويتضح ذلك من وجود إضافة إلى `runs/...` ونتائج تشغيل تحت `leverv2images` و `ElevatorLeverPose-2` ملفات ومجلدات مثل `runsposeleverv2modelweightsbest.pt` [file:1][code_file:2].

النسخة الثانية لم تكن مجرد إعادة تدريب، بل كانت تحسينًا عمليًا لاختيار أفضل وزن بين عدة نتائج مثل `runs/pose/train3/weights/best.pt` و `runs/pose/train4/weights/best.pt` ثم ربط الوزن الأنسب [file:1][code_file:2]. لاستخدامه وقت التشغيل الحقيقي `leverdetector.py` مباشرة داخل

المشكلات التي ظهرت والحلول

المشكلة	الوصف	الحل الذي تم اتباعه
bad magic/Failed to initialize device	عند Astra Pro فشل متكرر في تهيئة <code>orbbeccamera</code> [file:1][code_file:2]	تجربة point cloud، تعطيل FPS، تقليل الدقة و مختلف، ثم الانتقال إلى backend <code>astracamera</code> لـ Astra Pro [file:1]
astrapro.launch.py was not found	غير موجود داخل <code>launch</code> ملف [file:1][code_file:2]	العملية واستخدام <code>launch</code> مراجعة ملفات أو الانتقال لحزمة أخرى <code>astra.launch.py</code> [file:1] توفر الملف الصحيح
Unable to locate package ros-humble-astra-camera	كما هو <code>apt</code> الحزمة لم تكن متاحة عبر [file:1][code_file:2]	<code>clone</code> الانتقال إلى الثبيت من المصدر عبر <code>colcon build</code> ثم <code>rosdep install</code> [file:1]
Package astracamera not found	source قبل عمل <code>launch</code> تشغيل workspace صحيح للـ [file:1][code_file:2]	<code>source install/setup.bash</code> تنفيذ [file:1] عند الحاجة <code>bashrc</code> . وإضافته إلى
FileNotFoundError لملف الوزن	غير صحيح أو <code>best.pt</code> مسار الملف غير موجود في المكان المتوقع [file:1][code_file:2]	للبحث عن أفضل وزن فعلي <code>find</code> استخدام وتحديث المسار داخل الكود أو نسخ الوزن إلى المجلد المطلوب [file:1]
IndexError: index 0 is out of bounds	غير <code>keypoint</code> محاولة الوصول إلى موجودة عندما لا يعيد النموذج نقاطًا [file:1][code_file:2]	<code>keypoints</code> إضافة شرط يتحقق من وجود <code>xy[0][0]</code> [file:1] وطولها قبل الوصول إلى
libexec directory does not exist	خطأ ناتج عن إعدادات داخل <code>package/entry points</code> ROS2 [file:1][code_file:2]	بشكل صحيح <code>setup.py</code> و <code>setup.cfg</code> ضبط <code>resource files</code> و <code>executable</code> والتأكد من [file:1]
Package levervisionmodel not found	source الباكج لم يُنَّ أولم يتم عمل لها [file:1][code_file:2]	<code>colcon build</code> إعادة البناء بـ <code>source install/setup.bash</code> [file:1]
RViz Fixed Frame Error	<code>map</code> خطأ في الإطار المرجعي مثل <code>TF</code> مناسب عند غياب [file:1][code_file:2]	إلى Fixed Frame تغيير <code>base_link/baseLink</code> أو نشر <code>TF</code> مناسب [file:1]
عمق غير مستقر أو قيم صفرية	أو <code>blind spot</code> أو <code>depth</code> ضوءا IR انعكاسات [file:1]	<code>Spatial Median Filter</code> 5 بحجم 5x، <code>mm</code> - مثل -49 <code>calibration offset</code> وإضافة [file:1] والتعامل مع القيم الصفرية بعناية

أوامر الطرفية المستخدمة

أوامر إعداد البيئة وبناء النظام

- `cd ros2ws [file:1][code_file:2]`
- `rm -rf build install log [file:1]`
- `colcon build --symlink-install [file:1][code_file:2]`
- `colcon build --packages-select leverbvisionmodel [file:1][code_file:2]`
- `colcon build --packages-select astracamera [file:1][code_file:2]`
- `source install/setup.bash [file:1][code_file:2]`
- `echo "source ~/ros2ws/install/setup.bash" >> ~/.bashrc` ثم `source ~/.bashrc` كحل لتجنب نسيان الـ `source [file:1]`

أوامر تثبيت الاعتماديات

- `sudo apt update [file:1][code_file:2]`
- `rosdep update [file:1][code_file:2]`
- `rosdep install --from-paths src --ignore-src -r -y [file:1][code_file:2]`
- `pip install ultralytics [file:1][code_file:2]`

أوامر التعامل مع الكاميرا والدرافير

- `ros2 launch orbbeccamera astra.launch.py enable_point_cloud:=false` بصيغ متقاربة ظهرت `[file:1][code_file:2]` في المحادثة مع تغيير بارامترات الدقة والسرعة
- `ros2 launch orbbeccamera astra.launch.py depth_width:=320 depth_height:=240 color_width:=320 color_height:=240` بصيغ مكافئة لتخفيف الحمل `[file:1]`
- `ros2 launch astracamera astrapro.launch.py` أو الصيغة المقابلة `astrapro.launch.xml` حسب المرحلة `[file:1][code_file:2]` المسار المستخدم في تلك المرحلة
- `ros2 topic list` للتحقق من ظهور مواضيع الكاميرا `[file:1][code_file:2]`
- `sudo bash installudevrules.sh [file:1][code_file:2]`
- `sudo udevadm control --reload-rules [file:1][code_file:2]`
- `sudo udevadm trigger [file:1][code_file:2]`

أوامر إدارة الملفات والنماذج

- `find ros2ws/src/leverbvisionmodel/runs -name best.pt [file:1][code_file:2]`
- `ls ros2ws/src/leverbvisionmodel [file:1][code_file:2]`
- `cp ~/Downloads/best.pt ros2ws/src/leverbvisionmodel/ [file:1][code_file:2]`
- `chmod +x leverdetector.py [file:1][code_file:2]`

أوامر تشغيل النظام

- `python3 leverdetector.py` [file:1][code_file:2] لتشغيل المباشر في المراحل الأولى
- `python3 ros2ws/src/levervisionmodel/leverdetector.py` [file:1][code_file:2] عند الاختبار من المسار الكامل
- `ros2 launch levervisionmodel robotbringup.launch.py` [file:1][code_file:2] لتشغيل النسخة المدمجة داخل ROS2
- `ros2 run rviz2 rviz2` [file:1][code_file:2] والهدف TF والتحقق من RViz لفتح
- `ros2 run tf2_ros static_transform_publisher 0.2 0 0.3 0 0 0 baselink cameraLink` [file:1][code_file:2] مع متغيرات قريبة منها حسب المعايير الفعلية

ما تم تحسينه أثناء التطوير

إضافة، pose estimation إلى detection من أبرز التحسينات التي ظهرت خلال التقدم في المشروع: الانتقال من تعويض انحراف القياس، نقل النظام من سكرت calibration offset فلانر للعمق لتقليل الضوضاء، إدخال ديناميكي للهدف المكتشف حتى يمكن استهلاكه من طبقات TF متكاملة، ثم بث ROS2 package مستقل إلى [file:1]. التخطيط أو التحكم

launch وإضافة، resource و setup.py و setup.cfg كذلك تم تحسين قابلية التشغيل عبر تنظيم الملفات، وضبط OpenCV بإطار مرجعي واضح بدل الاكتفاء بعرض محلي عبر RViz موحدة، والتأكد من أن النتائج تظهر في file فقط [file:1].

الشكل النهائي للنظام

ونود رؤية، color/depth تنشر Astra Pro في صورته النهائية كما تعكسها المحادثة، أصبح النظام يتكون من كاميرا camera_link تربط TF ووحدة معالجة عمق مع فلترة ومعايرة، ثم طبقة YOLOv8-Pose تحمل نموذج ليصبح صالحًا للربط مع ذراع روبوتية أو نظام تحكم RViz و ROS2 داخل lever_target وتنشر موضع base_link و [file:1]. لاحق

هذا يعني أن المشروع لم يكن مجرد تجربة نموذج ذكاء اصطناعي، بل تطور إلى منظومة متكاملة تشمل الاستشعار، [file:1]. والإدراك، والتحويل الهندسي، والتصحيح، والتشغيل ضمن بنية روبوتية عملية